Two-week progress update

Building the Tonic experiment from the DSA lesson

Hongtao Zhang

Progress window: Apr 16–30, 2026

The simple story: earlier `dsa-stdexec` experiments showed how to test accelerator overhead by removing one software layer at a time. Over the last two weeks, I made the Rust IDXD path clean enough to run that style of experiment inside Tonic.

The big picture

What we learned before

1. DSA can be fast when submission cost is amortized.
2. Then the software path becomes visible.
3. The useful method was not guessing; it was comparing controlled layers.

What I did now

1. Cleaned up the Rust IDXD crates.
2. Built a clearer async memmove path.
3. Proved one DSA operation and one IAX operation on hardware.

What this enables

1. A Tonic experiment with the same discipline.
2. Ordinary path, software-control path, and IDXD path can be compared.
3. The claim can stay small until the data supports more.

Small claim for today: the Rust IDXD path is cleaner and has a limited real-hardware proof. I am not claiming a Tonic speedup yet.

Reminder: why the old DSA experiment mattered

Layer-removal result

Path	Throughput	Per-op	vs base
Full stdexec	26.3 Mpps	38.0 ns	1.00x
Direct path	41.6 Mpps	24.0 ns	1.58x
Reusable ops	59.9 Mpps	16.7 ns	2.28x

The key number

Removing framework layers cut per-operation cost by about `56%`: from `38.0 ns` to `16.7 ns`. At low concurrency, the reusable path reached `84 Mpps`.

Real hardware context

DSA lower bound

A later hardware-floor run reached `48.4 Mops/s` for `64 B` memmove with pipelined batching.

What that means

The device is not the only problem. Once hardware work is cheap enough, software structure can decide whether offload helps.

This is the method I want to reuse: build a ladder of comparable paths, remove one cost at a time, and only then say where the bottleneck is.

Reusing that method for Tonic

Experiment step	What `dsa-stdexec` did	Tonic version
1. Baseline	Full stdexec path with the normal sender/receiver stack.	Ordinary Tonic path with instrumentation off for throughput.
2. Software controls	Direct and reusable strategies removed framework costs one layer at a time.	Pooled buffers, copy-minimized paths, and lower-overhead stage counters.
3. Hardware path	Real DSA run checked whether the lower software cost transfers to hardware.	Prepared-host IDXD path through the same workload and artifact format.
4. Matched comparison	Same operation, size, concurrency, and strategy labels.	Same payload shape, size, concurrency, runtime, and endpoint split.

The next Tonic experiment should be a matched ladder: ordinary Tonic, software control Tonic, then IDXD Tonic.

What changed in the last two weeks

Cleaner measurement boundary

1. The current Tonic package now says plainly what it can and cannot prove.
2. The current rows do not show an IDXD win.
3. The workflow is still useful because it is rerunnable and reviewable.

Cleaner Rust path

1. `idxd-rust` is now the safe Rust crate.
2. `idxd-sys` is the raw UAPI/MMIO crate.
3. Async memmove now uses explicit owned buffers.

Small hardware proof

1. `IdxdSession<Dsa>` ran DSA memmove.
2. `IdxdSession<Iax>` ran IAX crc64.
3. A small release-mode benchmark produced positive rows.

I see this as preparation work for the right experiment, not as the final application result.

Tonic today: useful, but not a speedup claim

The current Tonic result is negative or inconclusive: the package proves the workflow, but the current rows do not show IDXD beating the ordinary path.

What is already useful

1. Software path validation exists.
2. IDXD verifier gate exists.
3. JSON / CSV / markdown comparison package exists.
4. Current rows show IDXD at roughly `0.003x–0.653x` of software throughput.

What is missing

1. A fresh prepared-host IDXD rerun.
2. Matched ordinary vs IDXD artifacts from the same run context.
3. Lower-overhead stage attribution.
4. A clear crossover rule for when offload helps.

How I would say this to an advisor

I do not want to sell a speedup. I want to say that the comparison machinery is now honest enough to run the next experiment cleanly.

Why the Rust cleanup matters for the experiment

Before

1. Old names like `dsa-ffi` and `idxd-bindings` made ownership unclear.
2. Async request helpers hid too much buffer behavior.
3. Some proof scripts and APIs were hard to explain as one clean path.

Now

1. `AsyncMemmoveRequest::new(source: Bytes, destination: BytesMut)` makes ownership explicit.
2. Direct Tokio completion owns accepted operations until completion.
3. Errors expose phase, retry, completion, and validation metadata without dumping payload bytes.

This matters because Tonic offload will fail if the memory ownership and completion path are unclear. The cleanup makes the next experiment easier to trust.

Small hardware proof through the new path

Evidence	Target	Device	Bytes	Iters	Mean latency	Rate
Operation proof	dsa-memmove	/dev/dsa/wq0.0	64	n/a	completed	pass
Operation proof	iax-crc64	/dev/iax/wq1.0	64	n/a	completed	crc ok
Small bench	dsa-memmove	/dev/dsa/wq0.0	4096	1000	6,837 ns	146,246 ops/s
Small bench	iax-crc64	/dev/iax/wq1.0	4096	1000	2,178 ns	459,064 ops/s

Verifier

pass

operation proof and small benchmark both passed

Build

release

measurements were collected in release mode

Failures

0 / 2000

benchmark operations completed without failed rows

> This is a proof that the path works for two representative operations. It is not a full performance study.

The next Tonic experiment I want to run

Matched workload ladder

1. Start with ordinary Tonic, instrumentation off.
2. Rerun the same points with software controls: pooled buffers and copy-minimized mode.
3. Rerun the same points through the IDXD path.
4. Compare only matched size, payload shape, concurrency, runtime, and endpoint setup.

Evidence to collect

1. Throughput and p99 from instrumentation-off runs.
2. Stage attribution from lower-overhead counters.
3. `perf` counters or flamegraphs for CPU explanation.
4. Artifact package with ordinary, software-control, and IDXD rows side by side.

This mirrors the old DSA experiment: baseline, remove software overhead, add real hardware, then compare matched rows.

Which Tonic points should be first?

4 KiB structured

The pooled-buffer control previously improved throughput by about `2.51x`.

Good first test for buffer policy.

1 MiB structured

The earlier run was strongly memory-bound: about `63.7%` memory-bound in the refined pass.

Good first test for copy movement.

64 KiB compression

Compression is payload-sensitive: structured was `0.585x`, random collapsed to `0.112x`.

Use only as a gated follow-up.

> My preferred first pass is `4 KiB` and `1 MiB` uncompressed. Compression should come after the copy/buffer story is clean.

My struggle with the agent

What the agent kept giving me

1. too many internal artifact names
2. too many verification details
3. a status report instead of a story

What I kept asking for

1. one small claim
2. human wording
3. connection to the old DSA experiment
4. a clear next experiment

> The real revision was turning implementation progress into a research story I can say out loud.

What I want feedback on

Decision question

Does this experiment ladder answer the advisor-level question: **when does the software path erase the benefit of accelerator offload?**

If yes, I should spend the next step on the prepared-host Tonic rerun.

Small conclusion

1. The old DSA result gave the method.
2. The last two weeks made the Rust IDXD path usable for that method.
3. The next experiment should decide whether the method carries into Tonic.

Bottom line: I am not presenting a Tonic speedup result yet. I am presenting a cleaner path to run the right experiment, with enough hardware proof to make that next experiment credible.